

EJADA EGYPT SUMMER INTERNSHIP PROGRAM 2026

Cloud Build Track — OCI & Terraform

Week 3 — Terraform Modules

Module Reference

modules/network, modules/subnet, modules/oke — Purpose, Files, Inputs, Outputs, and Root Wiring

Student	Salah Abdelhady
Working stack	terraform/nonprd/week3-containerized-oke/

Contents

1. Overview	3
2. Module: network	4
2.1 Purpose	4
2.2 Files	4
2.3 Inputs	4
2.4 Outputs — what to look for	4
2.5 How root calls it	5
3. Module: subnet	6
3.1 Purpose	6
3.2 Files	6
3.3 Inputs	6
3.4 Outputs — what to look for	7
3.5 How root calls it	8
4. Module: oke	9
4.1 Purpose	9
4.2 Files	9
4.3 Inputs	10
4.4 Outputs — what to look for	11
4.5 How root calls it	11
5. Network Topology at Runtime	13
6. Traffic Path	14
7. Kubernetes Workload Layer (k8s/, not Terraform)	15
8. Remote State (Object Storage)	16
9. Quick Reference — Root → Module Wiring	17
10. Related Documents	18

1. Overview

Week 3 Terraform is organized around three cohesive building blocks, each owned by exactly one module. Every resource that belongs together operationally sits in the same module call rather than being split across one-resource wrappers: the VCN and its gateways are one related group, each subnet is a complete related group in its own right, and the OKE cluster travels together with its node pool and NSGs.

Module	Related group	Root call
modules/network	VCN + Internet Gateway + NAT Gateway + Service Gateway + shared VCN flow-log group	module "network" once (network.tf)
modules/subnet	Subnet + route table + security list + optional flow log / capture filter	module "subnet" once with for_each = local.subnets (subnets.tf) — keys lb, workers, pods
modules/oke	OKE cluster + managed node pool + API / worker / pod NSGs	module "oke" once with count = var.enable_oke ? 1 : 0 (oke.tf)

Root locals.tf builds display names, tags, the security-list and route-table rule payloads, and the local.subnets map that drives module.subnet. Resource arguments that can change later come from var.* or local.* values, with the environment-specific values held in a gitignored terraform.tfvars. Child modules declare the oci provider requirement in their own versions.tf but inherit provider configuration from the root — there is no version pin or separate provider block inside any module.

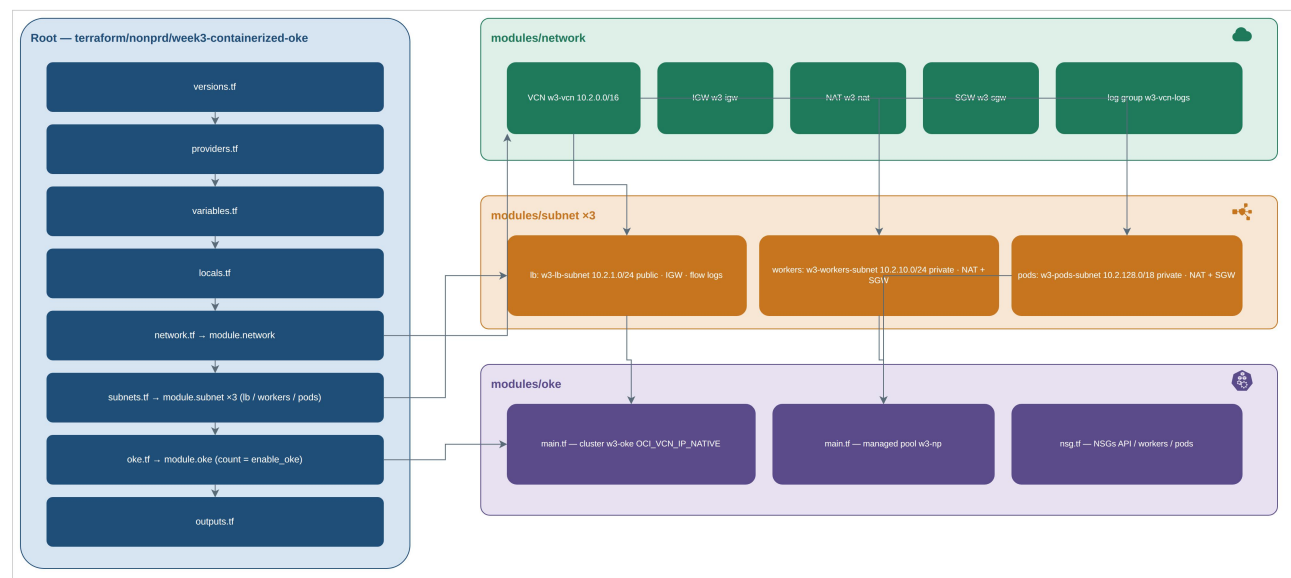


Figure 1. Terraform module map — root calls three related-group modules in sequence, with the application layer applied afterward.

Root calls three related-group modules in order: module.network (network.tf — VCN + Internet Gateway + NAT Gateway + Service Gateway + shared log group) → module.subnet x3 (subnets.tf — lb / workers / pods, each with its own route table, security list, and optional flow logs) → module.oke (oke.tf — cluster + VCN-native CNI + managed node pool; NSGs live in nsg.tf inside the same module). There is no root-level NSG file. The k8s/ manifests are applied with kubectl once the cluster exists; that layer is not a Terraform module and is covered in Section 7.

Remote state for this stack lives on OCI Object Storage's S3-compatible API, configured in backend.tf; the bucket itself is created by a sibling bootstrap-state stack. Section 8 covers the full remote-state wiring.

2. Module: network

Path: terraform/nonprd/week3-containerized-oke/modules/network/

2.1 Purpose

This is the VCN-core related group: the VCN itself plus the gateways and shared logging resource that every subnet depends on. Subnets — route table, security list, and optional per-subnet flow logs — live in modules/subnet. Cluster NSGs live in modules/oke. This module is called exactly once: one VCN, one Internet Gateway, one NAT Gateway, and, when enabled, one Service Gateway and one shared log group.

2.2 Files

File	What it declares
main.tf	data.oci_core_services.all_osn — lookup for “All Services In Oracle Services Network” (regex filter); locals pick local.osn_service from that data source. Resources: oci_core_vcn.this, oci_core_internet_gateway.this (always enabled), oci_core_nat_gateway.this, oci_core_service_gateway.this (count = var.enable_service_gateway ? 1 : 0, attaches All OSN via local.osn_service.id), oci_logging_log_group.vcn_flow (count = var.enable_log_group ? 1 : 0) — the shared group that subnet modules attach SERVICE flow logs to.
variables.tf	All module inputs — compartment, display names, CIDR, DNS label, enable flags, tags.
outputs.tf	VCN / gateway / log-group / OSN CIDR outputs for root and other modules.
versions.tf	required_providers for oracle/oci; no version pin; provider configuration inherited from root.
README.md	Short module contract mirroring this section.

2.3 Inputs

Variable	Description
compartment_ocid	Compartment OCID for the VCN, gateways, and shared log group.
vcn_display_name	VCN display name (e.g. w3-vcn).
vcn_cidr	IPv4 CIDR for the VCN.
vcn_dns_label	VCN DNS label (letters/digits, max 15).
igw_display_name	Internet Gateway display name.
nat_display_name	NAT Gateway display name.
sgw_display_name	Service Gateway display name.
enable_service_gateway	Create the Service Gateway (All OSN). Extra private-subnet SGW route rules are composed at the root (local.sgw_route), not inside this module.
enable_log_group	Create the shared VCN flow-log group.
log_group_display_name	Shared flow-log group display name.
freeform_tags / defined_tags	Tags applied to every resource in this module.

2.4 Outputs — what to look for

Output	Purpose / what to check after apply
vcn_id	VCN OCID. Passed to module.subnet (vcn_id) and module.oke (vcn_id). Expect a valid ocid1.vcn...

Output	Purpose / what to check after apply
internet_gateway_id	IGW OCID. Root local.gateway_ids.igw — default-route target for the public lb subnet.
nat_gateway_id	NAT OCID. Root local.gateway_ids.nat — default-route target for private workers / pods subnets.
service_gateway_id	SGW OCID when enable_service_gateway is true; otherwise null. Used as network_entity_id in local.sgw_route.
log_group_id	Shared flow-log group OCID when enable_log_group is true; otherwise null. Passed to every module.subnet call as log_group_id.
osn_cidr	All-OSN service CIDR block. Consumed by root local.sgw_route as the SGW route destination (SERVICE_CIDR_BLOCK).

After apply: one VCN, one IGW, one NAT, and — when the flags are on — one SGW and one log group. Never more than one of each.

2.5 How root calls it

network.tf calls the module once, with no count or for_each. Stage A can apply network-only while enable_oke is false; the VCN still exists on its own.

```

module "network" {
  source = "../modules/network"

  compartment_ocid      = var.compartment_ocid
  vcn_display_name      = local.vcn_display_name
  vcn_cidr               = var.vcn_cidr
  vcn_dns_label         = var.vcn_dns_label
  igw_display_name      = local.igw_display_name
  nat_display_name      = local.nat_display_name
  sgw_display_name      = local.sgw_display_name
  enable_service_gateway = var.enable_service_gateway
  enable_log_group      = local.any_subnet_logs # true if any subnet
                                              # has enable_logs
  log_group_display_name = local.log_group_display_name
  freeform_tags         = local.freeform_tags
  defined_tags          = local.defined_tags
}

```

local.any_subnet_logs is anytrue([for s in var.subnets : s.enable_logs]), so the shared log group is created when at least one subnet enables flow logs — in the lab design, that is the lb subnet.

3. Module: subnet

Path: terraform/nonprd/week3-containerized-oke/modules/subnet/

3.1 Purpose

This is the subnet related group: one subnet plus its own route table, security list, and optional VCN flow-log stack (capture filter, optional local log group, and a SERVICE log). The module is generic — ingress, egress, and route rules are rendered from input lists through dynamic blocks, so the same module code produces the public lb subnet and the private workers / pods subnets without any per-type branching inside the module.

Root calls it once with `for_each = local.subnets` (keys lb, workers, pods) — three subnet instances from a single module call.

3.2 Files

File	What it declares
main.tf	Locals for derived display names and log-group / capture-filter ID resolution. <code>oci_core_security_list.this</code> — dynamic "ingress_security_rules" / dynamic "egress_security_rules" over input lists, with nested dynamic <code>tcp_options</code> / <code>udp_options</code> / <code>icmp_options</code> when protocol and ports/types are set. <code>oci_core_route_table.this</code> — dynamic "route_rules" for the default 0.0.0.0/0 gateway (when <code>default_route_gateway_id</code> is non-null) plus another dynamic "route_rules" for extra_route_rules (e.g. a Service Gateway route). <code>oci_core_subnet.this</code> — attaches its own route table and security list. Optional logging: <code>oci_logging_log_group.this</code> (count when <code>enable_logs</code> && <code>create_log_group</code>), <code>oci_core_capture_filter.flow</code> (count when logs are on and <code>create_capture_filter</code> is true), <code>oci_logging_log.flow</code> (count when <code>enable_logs</code>) — a SERVICE flow log on the subnet, optionally referencing the capture filter.
variables.tf	All inputs: identity, CIDR, public/private flag, route-table and security-list rules, tags, and the optional flow-log knobs.
outputs.tf	Subnet, route-table, security-list, and log-related outputs.
versions.tf	oracle/oci provider requirement; inherits root provider configuration.
README.md	Short module contract.

Design notes encoded in the module

- `create_log_group` is a known bool input — never derived from `log_group_id`, since that OCID is unknown until apply.
- When root uses a shared log group: `create_log_group = false` and `log_group_id = module.network.log_group_id`.

3.3 Inputs

Variable	Description
<code>compartment_ocid</code>	Compartment for the subnet, route table, security list, and optional logs.
<code>vcn_id</code>	Owning VCN OCID (<code>module.network.vcn_id</code>).
<code>display_name</code>	Subnet display name; route-table and security-list names derive from it unless overridden.
<code>cidr_block</code>	Subnet IPv4 CIDR.
<code>dns_label</code>	Subnet DNS label (max 15).

Variable	Description
prohibit_public_ip_on_vnic	true = private; false = public.
route_table_display_name	Override; empty → <display_name>-rt.
security_list_display_name	Override; empty → <display_name>-sl.
default_route_gateway_id	IGW or NAT OCID for 0.0.0.0/0; null skips the default route.
default_route_destination	Default route destination CIDR (default 0.0.0.0/0).
default_route_description	Description on the default route rule.
extra_route_rules	Extra route-table rules — a list of objects: destination, destination_type, network_entity_id, description. Empty means none.
ingress_security_rules	Ingress list rendered with dynamic blocks (protocol 1 / 6 / 17 / all, optional min/max, icmp_type/code).
egress_security_rules	Egress list rendered with dynamic blocks.
freeform_tags / defined_tags	Tags on subnet-related resources.
enable_logs	Create a SERVICE flow log (and capture filter when configured).
create_log_group	Create a log group inside this module call; must be a known bool.
log_group_id	Existing log group OCID when create_log_group is false.
log_group_display_name	Name used when this call creates a group; empty → <display_name>-logs.
log_display_name	Flow log name; empty → <display_name>-flow-log.
log_retention_duration	Retention days (30 / 60 / 90 / 120 / 150 / 180).
log_category / log_service	Logging source category / service (defaults all / flowlogs).
create_capture_filter	Create oci_core_capture_filter and attach it to the flow log.
capture_filter_display_name	Override; empty → <display_name>-flow-filter.
flow_log_sampling_rate	Sampling rate (1, 10, 50, or 100).
capture_filter_id	Existing capture filter when create_capture_filter is false.

3.4 Outputs — what to look for

Output	Purpose / what to check after apply
subnet_id	Subnet OCID. Consumed by module.oke as endpoint_subnet_id / worker_subnet_id / pod_subnet_ids / service_lb_subnet_id via module.subnet["lb" "workers" "pods"].subnet_id.
subnet_cidr	Echo of the subnet CIDR — useful when cross-checking NSG or security-list sources.
route_table_id	OCID of this subnet's route table.
security_list_id	OCID of this subnet's security list.
log_group_id	Log group actually in use (created here or passed in); null when logs are off.
log_id	Flow log OCID when enable_logs is true; else null.
capture_filter_id	Capture filter in use when flow logs are enabled; else null.
prohibit_public_ip_on_vnic	Echo of the public vs private setting, for quick verification.

After apply with the lab map: three instances — `module.subnet["lb"]`, `module.subnet["workers"]`, `module.subnet["pods"]` — each with its own route table and security list. Flow log and capture filter attach on the key(s) where `enable_logs` is true (lab: lb only), using the shared log group from `module.network`.

3.5 How root calls it

```

module "subnet" {
  source      = "../modules/subnet"
  for_each    = local.subnets

  compartment_ocid      = var.compartment_ocid
  vcn_id                 = module.network.vcn_id
  display_name           = each.value.display_name
  cidr_block             = each.value.cidr
  dns_label              = each.value.dns_label
  prohibit_public_ip_on_vnic = each.value.prohibit_public_ip_on_vnic
  default_route_gateway_id = each.value.default_route_gateway_id
  default_route_description = each.value.default_route_description
  extra_route_rules       = each.value.extra_route_rules
  ingress_security_rules  = each.value.ingress_security_rules
  egress_security_rules   = each.value.egress_security_rules
  enable_logs             = each.value.enable_logs
  create_log_group        = false
  log_group_id            = module.network.log_group_id
  log_retention_duration  = var.log_retention_duration
  flow_log_sampling_rate  = var.flow_log_sampling_rate
  freeform_tags           = local.freeform_tags
  defined_tags            = local.defined_tags
}

```

`local.subnets` (in `locals.tf`) is built from `var.subnets`. Keys stay `lb` / `workers` / `pods` so `module.oke` can look them up by name. `default_route_gateway_id` resolves to `local.gateway_ids[s.route_type]` (igw or nat, sourced from `module.network`). `extra_route_rules` resolves to `local.sgw_route` for NAT-routed subnets when the Service Gateway is enabled, and is empty for the IGW-routed `lb` subnet. `ingress_security_rules` and `egress_security_rules` come from `local.subnet_ingress` and `local.default_egress` (ICMP, API, SSH, kubelet, NodePort, `workers`↔`pods`, and so on).

Key	Role
lb	Public (IGW). Kubernetes API endpoint + Service LBs. Flow logs enabled in the lab map.
workers	Private (NAT). Managed node pool VNICs.
pods	Private (NAT), typically a large CIDR (e.g. /18). VCN-native CNI assigns each pod a VCN IP from this subnet.

4. Module: oke

Path: terraform/nonprd/week3-containerized-oke/modules/oke/

4.1 Purpose

This is the OKE related group: the container engine cluster, its managed node pool, and the optional API / worker / pod Network Security Groups attached to them. NSGs stay inside this module (nsg.tf) because they belong to the same cluster building block — not a separate one-resource module.

Root calls the module once behind `count = var.enable_oke ? 1 : 0`, so Stage A can apply network-only. Workloads stay in k8s/ (kubectl, once the cluster is ACTIVE) — there is no Kubernetes provider in this stack.

Lab CNI: OCI_VCN_IP_NATIVE (each pod gets a VCN IP from the pod subnet). The overlay pods_cidr input is ignored while native CNI is active.

4.2 Files

File	What it declares
main.tf	Locals: <code>is_native</code> (<code>cni_type == "OCI_VCN_IP_NATIVE"</code>), <code>is_flex_shape</code> (shape name contains FLEX). <code>oci_containerengine_cluster.this</code> — cluster type, Kubernetes version, <code>cluster_pod_network_options</code> CNI, <code>endpoint_config</code> (public IP flag, endpoint subnet, API NSG IDs), <code>options.service_lb_subnet_ids</code> , add-ons, <code>kubernetes_network_config</code> (services CIDR; <code>pods_cidr</code> only when not native). Lifecycle preconditions: a non-empty Kubernetes version; native CNI requires <code>pod_subnet_ids</code> . <code>oci_containerengine_node_pool.this</code> — size, worker NSGs, placement AD + worker subnet, <code>node_pool_pod_network_option_details</code> (CNI, max pods, pod subnet IDs, pod NSGs when native), dynamic " <code>node_shape_config</code> " when flex, <code>node_source_details</code> (image + boot volume). Lifecycle preconditions: node image, worker subnet, SSH public key.
nsg.tf	Intentional for_each design for NSGs and rules — see the table below.
variables.tf	Cluster / node-pool / NSG / port / CIDR inputs.
outputs.tf	Cluster, node pool, endpoints, and NSG OCIDs.
versions.tf	oracle/oci provider requirement; inherits root provider configuration.
README.md	Short module contract.

nsg.tf — for_each design

NSGs and security rules use maps and for_each so api / workers / pods never appear as copy-pasted, near-identical resource blocks. The four rule resources below all share the `oci_core_network_security_group_security_rule` prefix; only the resource name after the dot is shown.

Resource	Pattern
<code>oci_core_network_security_group.this</code>	for_each over <code>local.nsg_display_names</code> (api, workers, and pods only when VCN-native). Empty map when <code>enable_nsgs</code> is false.
<code>.api_ingress_tcp</code>	for_each over <code>local.api_ingress_tcp</code> (kubectl, workers→API, pods→API, the OKE control port).
<code>.workers_ingress_tcp</code>	for_each over <code>local.worker_ingress_tcp</code> (SSH, kubelet, the app port, the NodePort range).
<code>.all_ingress</code>	for_each over <code>local.nsg_all_ingress</code> — workers↔pods “allow all from subnet” rules from one map, with the source flipped per entry.

Resource	Pattern
.egress_all	for_each over local.nsg_egress_all — the same allow-all egress shape on every NSG; only the description differs.

Locals also expose `nsg_api_ids` / `nsg_worker_ids` / `nsg_pod_ids` lists, attached on the cluster endpoint, node pool, and (native) pod network options respectively.

4.3 Inputs

Variable	Description
compartment_ocid	Compartment for the cluster, node pool, and optional NSGs.
vcn_id	Hosting VCN OCID.
cluster_name	OKE cluster display name.
cluster_type	BASIC_CLUSTER (lab) or ENHANCED_CLUSTER.
kubernetes_version	Version string (e.g. v1.31.1).
cni_type	Pod CNI; lab uses OCI_VCN_IP_NATIVE.
endpoint_public	Public IP on the Kubernetes API endpoint.
endpoint_subnet_id	API endpoint subnet (module.subnet["lb"].subnet_id).
worker_subnet_id	Worker subnet OCID.
pod_subnet_ids	Pod subnet OCIDs (required for VCN-native).
service_lb_subnet_id	Public subnet for Kubernetes Service load balancers.
services_cidr	Kubernetes Service CIDR (must not overlap the VCN).
pods_cidr	Overlay pod CIDR; used only when the CNI is FLANNEL_OVERLAY.
dashboard_enabled	OKE Kubernetes dashboard add-on.
availability_domain	AD for node-pool placement.
node_pool_name	Managed node pool display name.
node_pool_size	Worker count.
node_shape	Worker shape.
node_ocpus / node_memory_in_gbs	Flex shape sizing (via dynamic "node_shape_config").
node_boot_volume_size_in_gbs	Worker boot volume size.
node_image_id	OKE-compatible node image OCID.
ssh_public_key	SSH public key on workers (sensitive).
max_pods_per_node	Max pods per worker (VCN-native; limited by the shape's VNIC count).
freeform_tags / defined_tags	Tags on OKE resources.
enable_nsgs	Create and attach API / worker / pod NSGs.
nsg_api_display_name / nsg_workers_display_name / nsg_pods_display_name	NSG display names.
worker_subnet_cidr / pod_subnet_cidr / lb_subnet_cidr	CIDRs used as NSG rule sources.
allowed_api_cidr	Client CIDR allowed to reach the Kubernetes API (prefer YOUR_IP/32).

Variable	Description
api_port	Kubernetes API TCP port (default 6443).
oke_control_port	OKE control/proxy port (default 12250).
kubelet_port	Kubelet port on workers (default 10250).
app_port	App / load-balancer target port (default 80).
node_port_min / node_port_max	NodePort range for Service LB traffic (defaults 30000–32767).

4.4 Outputs — what to look for

Output	Purpose / what to check after apply
cluster_id	Cluster OCID — use with oci ce cluster create-kubeconfig; also surfaced as a root output.
cluster_name	Cluster name for Console/CLI confirmation.
node_pool_id	Managed node pool OCID for oci ce node-pool lookups.
kubernetes_version	Version applied on the cluster — compare to var.kubernetes_version / local.kubernetes_version.
cni_type	CNI configured on the cluster (echo of the input).
endpoints	Public/private endpoint map when populated; the public endpoint is how kubectl reaches the API from outside the VCN.
nsg_api_id	API NSG OCID when enable_nsgs is true; else null.
nsg_workers_id	Worker NSG OCID when enabled; else null.
nsg_pods_id	Pod NSG OCID when VCN-native NSGs are enabled; else null.

After apply (with enable_oke = true): one ACTIVE cluster, one node pool of size var.oke_node_pool_size, and — when NSGs are enabled — exactly one API NSG, one worker NSG, and (VCN-native) one pod NSG, each with its rule set from the for_each maps above. Address the module instance as module.oke[0] because of count.

4.5 How root calls it

oke.tf also declares identity and catalog data sources used by root locals (not inside the module):

- data.oci_identity_availability_domains.ads — AD for placement (local.availability_domain).
- data.oci_containerengine_cluster_option.all (count = var.enable_oke ? 1 : 0) — Kubernetes version catalog; local.kubernetes_version prefers var.kubernetes_version or the latest from the catalog.
- data.oci_containerengine_node_pool_option.all — node image catalog; local.oke_node_image_id prefers var.oke_node_image_id or a filtered OKE image from the catalog.

```
module "oke" {
  count = var.enable_oke ? 1 : 0
  source = "../modules/oke"

  compartment_ocid = var.compartment_ocid
  vcn_id            = module.network.vcn_id
  cluster_name     = local.oke_cluster_display_name
  cluster_type     = var.oke_cluster_type
  kubernetes_version = local.kubernetes_version
  cni_type         = var.oke_cni_type
  endpoint_public  = var.oke_endpoint_public
  endpoint_subnet_id = module.subnet["lb"].subnet_id
  worker_subnet_id = module.subnet["workers"].subnet_id
  pod_subnet_ids   = [module.subnet["pods"].subnet_id]
```

```
service_lb_subnet_id      = module.subnet["lb"].subnet_id
# ... services/pods CIDR, dashboard, AD, node pool sizing,
# ... image, SSH key, tags ...
enable_nsgs               = var.enable_oke_nsgs
# ... NSG display names, subnet CIDRs, ports from var/local ...
depends_on                 = [module.subnet]
}
```

Subnet wiring: the API endpoint and the Service load balancer share the public lb subnet; workers use the workers subnet; pods use the pods subnet as a single-element list for VCN-native CNI.

5. Network Topology at Runtime

The three modules above combine into a single physical picture once applied. This is the shape the previous sections' inputs and outputs actually produce inside the VCN.

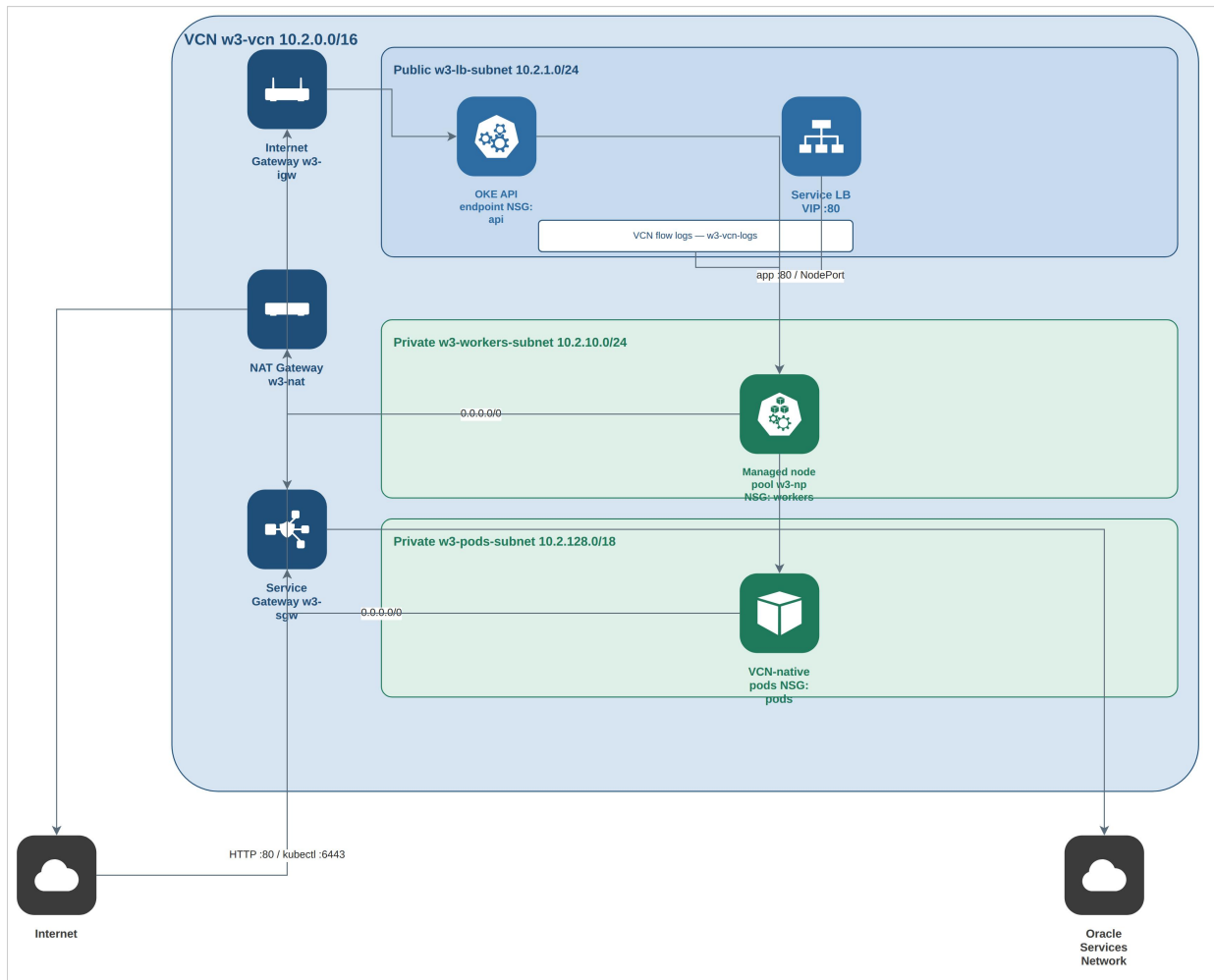


Figure 2. Network topology — three subnets, gateways, flow logs, and the NSGs attached to each layer.

Three subnets carry the whole stack: public 10.2.1.0/24 (the OKE API endpoint and the Service load balancer, routed through the Internet Gateway, with flow logs enabled), private workers 10.2.10.0/24, and private pods 10.2.128.0/18 (both routed through the NAT Gateway, plus a Service Gateway route to the Oracle Services Network). Security lists sit on every subnet from modules/subnet; OKE NSGs from modules/oke attach separately to the API endpoint, the workers, and the VCN-native pods. Workers carry no public IP at all — every path in from the Internet terminates at the public subnet.

6. Traffic Path

This is the request path a client actually takes through the resources the modules created, and where the persistent volume sits relative to that path.

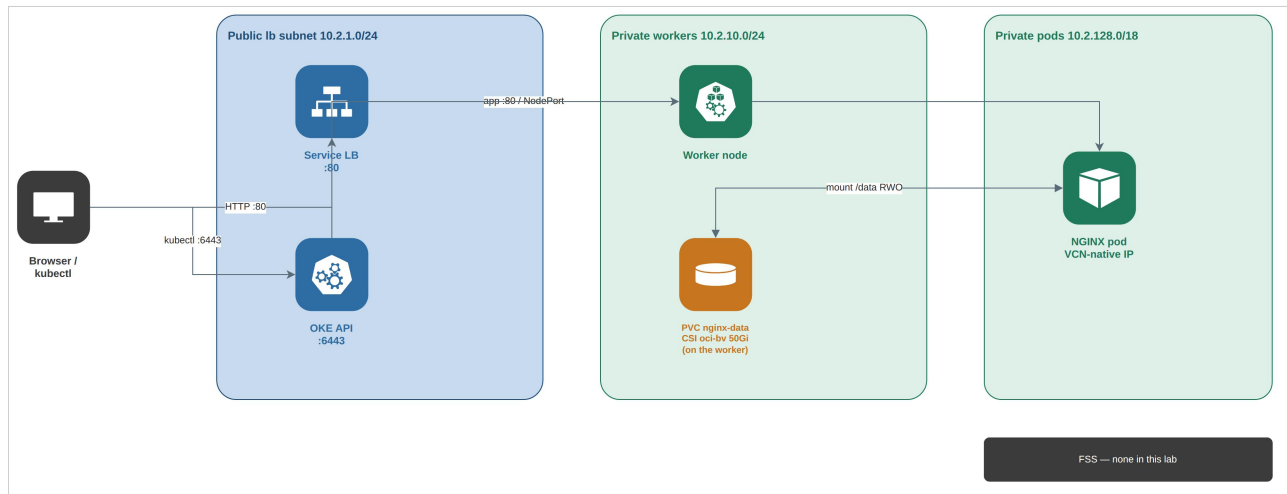


Figure 3. Traffic path — the Internet reaches only the public lb subnet; storage is CSI oci-bv, not File Storage.

The Internet reaches only the public lb subnet: HTTP to the Kubernetes Service load balancer, and kubectl to the public OKE API. The Service load balancer forwards to the worker; kube-proxy delivers the request to VCN-native pods living in 10.2.128.0/18 — not a Flannel overlay network. Storage is CSI oci-bv: a block volume attached to the worker that runs the pod. There is no File Storage (FSS) anywhere in this lab.

Historical values from the lab run: demo VIP 193.122.68.107 after apply, worker private IP 10.2.10.193. The stack was destroyed after evidence capture; see the Week 3 Lab 3 Terraform Documentation for the full command history.

7. Kubernetes Workload Layer (k8s/, not Terraform)

Once the cluster reaches ACTIVE, the workload layer is applied with kubectl — there is no Kubernetes provider block anywhere in this stack, and none of the objects below are Terraform resources or show up in terraform state.

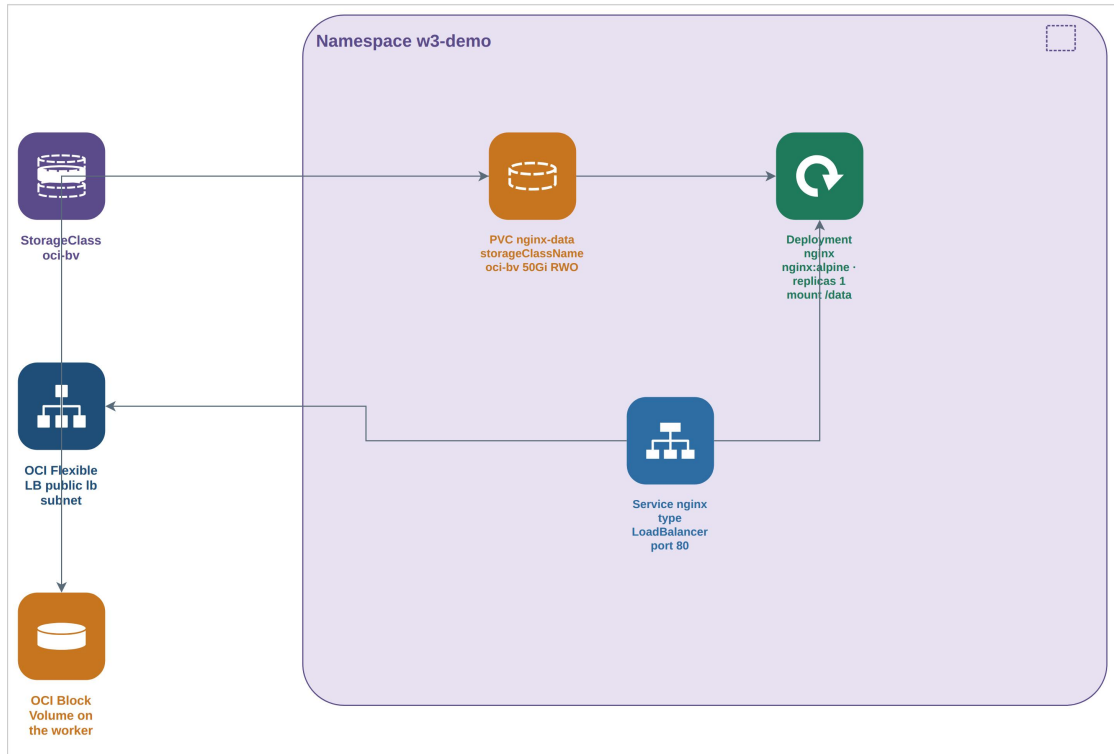


Figure 4. Kubernetes workload — StorageClass, PVC, Deployment, and Service inside namespace w3-demo.

Apply order: namespace w3-demo, StorageClass oci-bv, PVC nginx-data, Deployment nginx, then Service nginx (type: LoadBalancer). The image must be fully qualified for OKE's CRI-O runtime — a bare nginx:alpine tag is not enough. The PVC provisions an OCI Block Volume and attaches it to the worker; NGINX mounts it at /data. The Service provisions its own OCI Flexible Load Balancer in the public lb subnet, separate from (and downstream of) the OKE API endpoint that Terraform's module.oke exposes.

Object	Role
Namespace w3-demo	Scope for every object below; created first.
StorageClass oci-bv	blockvolume.csi.oraclecloud.com provisioner; the cluster ships this class by default, so this manifest documents it explicitly rather than creating something new.
PVC nginx-data	50Gi, ReadWriteOnce, storageClassName: oci-bv; provisions the Block Volume.
Deployment nginx	Image docker.io/library/nginx:alpine, replicas 1, mounts the PVC at /data.
Service nginx	type: LoadBalancer, port 80; provisions the OCI Flexible Load Balancer that the public lb subnet carries.

8. Remote State (Object Storage)

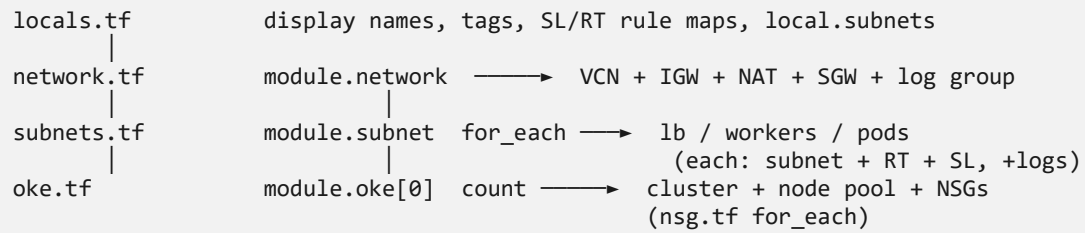
The Week 3 stack is configured for remote Terraform state on OCI Object Storage, accessed through its S3-compatible API.

Piece	Role
terraform/nonprd/bootstrap-state/	One-time stack that creates the Object Storage bucket and exposes namespace / endpoint outputs. This stack keeps local state by design — the bucket must exist before anything can store state in it.
week3-containerized-oke/backend.tf	terraform { backend "s3" { ... } } pointing at bucket w-terraform-state-intern18, key week3-containerized-oke/terraform.tfstate, region me-jeddah-1, with the skip_* and force_path_style flags required for OCI's S3-compatible endpoint. Backend blocks accept literals only — no variables.
backend.hcl.example	Template for passing backend settings via terraform init -backend-config=... when that is preferred over editing literals directly in backend.tf.

Credentials for the S3 backend are an OCI Customer Secret Key exposed as the `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` environment variables, and are never committed. Full operator setup steps live in the stack README and in `../bootstrap-state/README.md`.

9. Quick Reference — Root → Module Wiring

A compact, at-a-glance version of Figure 1, for scanning without opening the full diagram.



10. Related Documents

This reference covers the three Terraform modules only — their purpose, files, inputs, outputs, and how the root stack wires them together. The full lab narrative lives elsewhere:

- Week 3 Lab 3 Terraform Documentation — architecture, evidence, screenshots, the kubectl workload walkthrough, and the destroy procedure.

Working stack (source-controlled): github.com/Salahmohameds/ejada-oci-week1-terraform

End of the Terraform modules reference for Week 3.